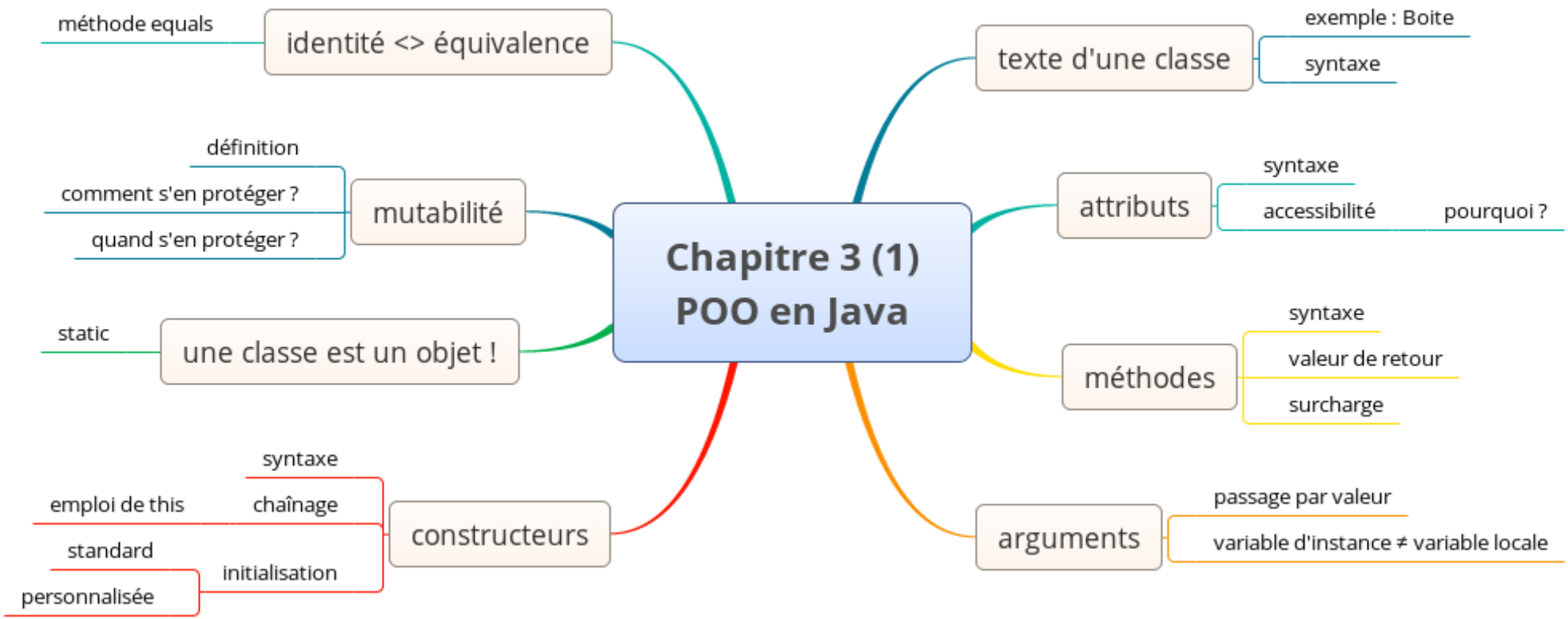
Séance 3 / 12


Chapitre 3 (1)
POO en Java
identité <> équivalence
méthode equals
mutabilité
définition
comment s'en protéger ?
quand s'en protéger ?
une classe est un objet !
static
constructeurs
syntaxe
chaînage
emploi de this
initialisation
standard
personnalisée
texte d'une classe
exemple : Boite
syntaxe
attributs
syntaxe
accessibilité
pourquoi ?
méthodes
syntaxe
valeur de retour
surcharge
arguments
passage par valeur
variable d'instance ≠ variable locale

Exemple de classe : Boite

```java
public class Boite {

  // ATTRIBUTS
  private boolean fermee;
  private String  nom;
  private int     valeur;

  // CONSTRUCTEURS
  public Boite(String n, int v) {
      this.nom = n;
      this.valeur = v;
      this.fermee = true;
  }
  public Boite(int v) {
      this("Standard", v);
  }

  // REQUÊTES
  public String nom() {
      return this.nom;
  }
  public int contenu() {
      return this.valeur;
  }
  public boolean fermee() {
      return this.fermee;
  }

  // COMMANDES
  public void remplir(int v) {
      if (this.fermee()) {
          throw new AssertionError();
      }
      this.valeur = v;
  }
  public void ouvrir() {
      this.fermee = false;
  }
  public void fermer() {
      this.fermee = true;
  }
}
```

Déclaration de classe (simplifiée)

```
class <IdClasse> {
    // Attributs
    [<visibilité>] [final] <type> <idChamp> [= <expr>] { , <idChamp> [= <expr>] }*
    // Constructeurs
    [<visibilité>] <IdClasse> ( [<args>] ) <bloc>
    // Méthodes
    [<visibilité>] {<Type> | void} <idMéthode> ( [<args>] ) <bloc>
}
```

```
<visibilité> ::= public | protected | private

<args> ::= <Type> <idArg> { , <Type> <idArg> }*
```

Attribut

```
[<visibilité>] [final] <Type> <id>
```

- *Attribut* (variable d'instance)
  - définit un champ pour les instances de la classe
    - `<visibilité>` **::=**

      `private` | **`protected`** | **`public`**

accessible partout

accessible uniquement dans le texte de la classe

```
// ATTRIBUTS
private boolean fermee;
private String  nom;
private int     valeur;
```

- Attribut préfixé de **`final`**
  - valeur constante

Pourquoi l'accessibilité ?

- Seuls appels autorisés = appels accessibles
  - **public**
    - accessible dans tout texte
  - **private**
    - accessible dans le texte de la classe uniquement

```
class Boite {
    private String mdp;
    public void vider(String motDePasse) {
        if (mdp.equals(motDePasse) {
            if (fermee()) {
                ouvrir();
            }
            valeur = 0;
            fermer();
        } else {
            System.out.println("Hé, hé !");
        }
    }
    ...
}
```

```
class Voleur {
    public voler(Boite b) {
        // Hu, hu !
        b.vider(b.mdp);
    }
}
```

Méthode

```
[<visibilité>] <Type> <id>([<args>]) [<ClauseErr>] <corps>
```

- Méthode = Entête + Corps
- *Entête* = Visibilité + Signature + ClauseErreur
- *Signature* = Type + Nom + (Args)
  - indique comment appeler la méthode
- *Corps* = bloc d'instructions
  - actions réalisées par l'objet lors de l'appel

```
public void ouvrir() {
    this.fermee = false;
}
```

Type de la valeur retournée par une méthode

- Devant le nom de la méthode

`valeur` est de type **`int`**

donc

```
public int contenu() {
    return valeur;
}
```

- **`void`** : pas de valeur retournée

```
public void fermer() {
    fermee = true;
    [return;]
}
```

Surcharge de méthode

- *Méthode surchargée*
  - définie plusieurs fois avec le même nom
  - seule sa liste d'arguments change

```
public void voler(Aile gauche, Aile droite) {
    // voler avec deux ailes
}
public void voler() {
    // voler comme SuperMan
}
```

- surcharge → même sémantique

```
public void voler(Aile gauche, Aile droite) {
    // battre des ailes pour s'élever en l'air
}
public void voler() {
    // dérober quelquechose à quelqu'un
}
```

- Rappel : on peut surcharger un constructeur (`Boite(String, int)` et `Boite(int)`)

Arguments

- Chaque argument déclaré avec son type

```
public StringBuffer replace(int start, int end, String str)
```

- Pas d'argument → parenthèses vides

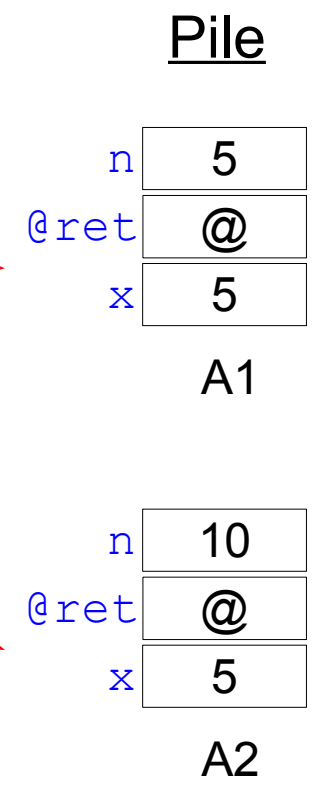
```
public void ouvrir()
```

- Passage d'arguments en Java
  - par **valeur** uniquement
  - paramètre formel = variable locale

Passage par valeur : types primitifs

```
void sansEffetDeBord(int n) {
    // A1
    n = 2 * n;
    // A2
    System.out.println(n);
}
```

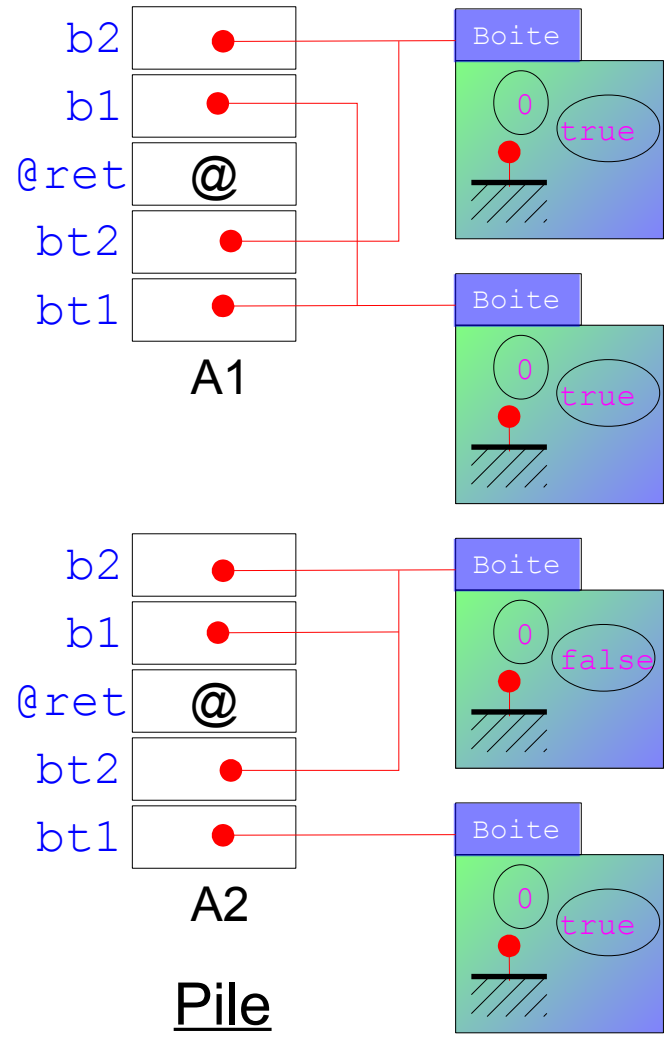
```
int x = 5;
sansEffetDeBord(x);     // 10
System.out.println(x); // 5
```



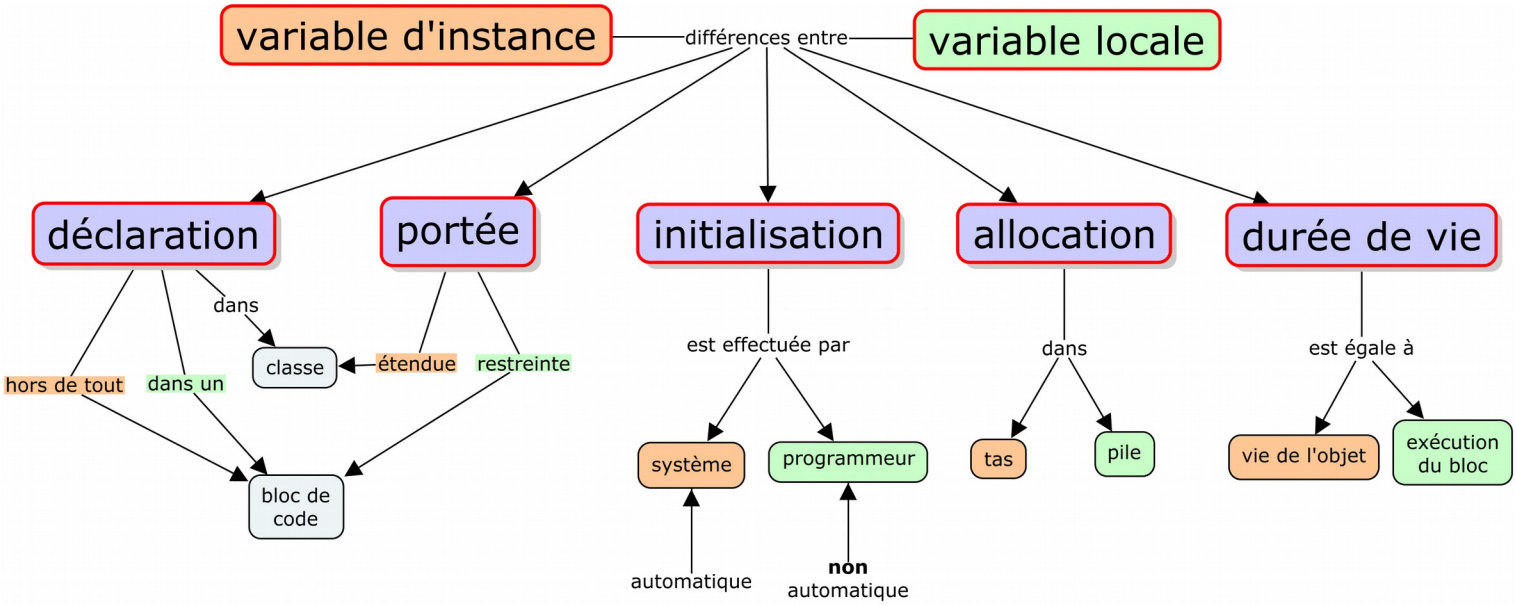
Passage par valeur : types d'objet

```
void effetDeBord(Boite b1, Boite b2) {
    // A1
    b1 = b2;
    b1.ouvrir();
    // A2
}
```

```
Boite bt1 = new Boite();
Boite bt2 = new Boite();
System.out.println(bt1.fermee()); // true
System.out.println(bt2.fermee()); // true
effetDeBord(bt1, bt2);
System.out.println(bt1.fermee()); // true
System.out.println(bt2.fermee()); // false
```



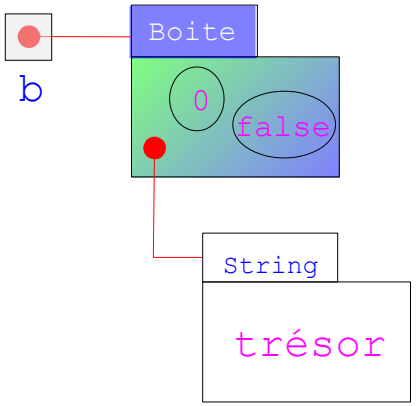
Variable d'instance ≠ Variable locale



Constructeur

- Procédure définie dans une classe
  - exactement le même nom que la classe
  - pour initialiser les instances

```
Boite b = new Boite("trésor", 0);
```



constructeur de la classe `Boite`

paramètres permettant d'initialiser l'état de l'objet

```
[<visibilité>] <IdClasse>([<args>]) [<ClauseErr>] <corps>
```

- `<ClauseErr>` **::=**

  `throws <TypeException>`

- `<corps>` **::=**

  `{ <code> }`

- *Constructeur*
  - s'utilise **uniquement** avec `new`
  - ne contient **jamais** d'instruction `return <expr>;`
  - peut contenir des instructions `return;`
  - **n'est pas** une méthode

- Plusieurs constructeurs (surcharge)
  - tous le même nom
  - distingués par leur séquence d'arguments

```
public Boite(String n, int v) {...}
public Boite(int v) {...}
```

- Un constructeur peut en appeler un autre de la même classe

il faut coder
`this(...);`
en première ligne

```
public Boite(int v) {
    this("Standard", v);
    // la suite à partir
    // d'ici
}
```

Création d'un objet en Java

```
new <IdClasse>([<args>])
```

effet de **new**

- Création d'une instance de `<IdClasse>` :
  - allocation mémoire
  - création de la référence
- Initialisation standard des champs :

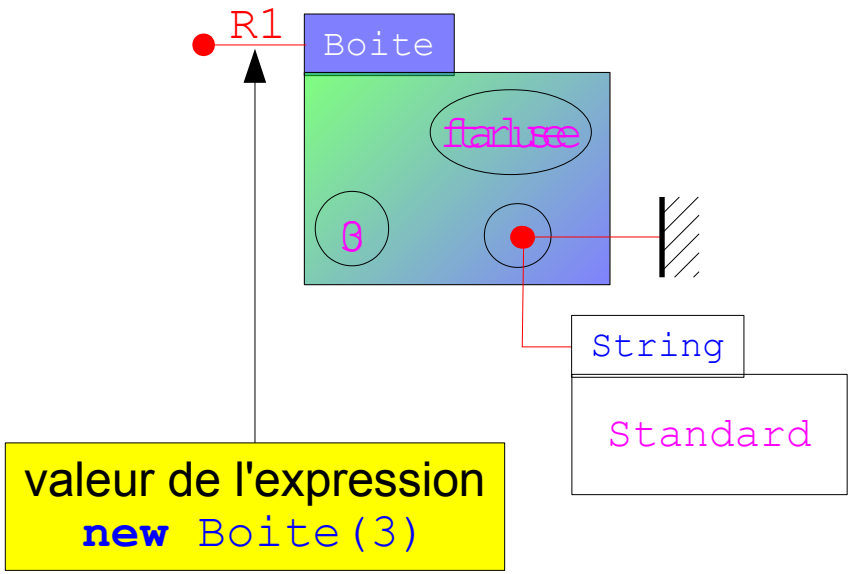
  numérique : `0`
  caractère : `'\u0000'`
  booléen : **`false`**
  référence : **`null`**

constructeur

- Personnalisation des champs
- Retour de la référence

- Exemple : évaluation de **new** `Boite(3)`
  - 1) allocation mémoire + création de la référence
  - 2) initialisation standard
  - 3) exécution successive des constructeurs
  - 4) retour de la référence du nouvel objet

```
class Boite {
    // ATTRIBUTS
    private boolean fermee;
    private String  nom;
    private int     valeur;
    // CONSTRUCTEURS
    public Boite(String n, int v) {
        this.nom = n;
        this.valeur = v;
        this.fermee = true;
    }
    public Boite(int v) {
        this("Standard", v);
    }
    ...
}
```

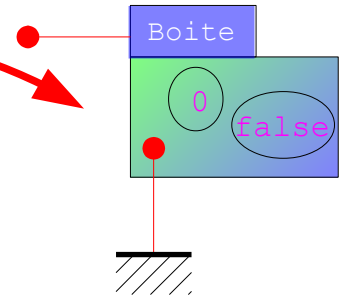


Classe sans constructeur explicite

- Pas de constructeur → constructeur standard

```
class Boite {
    // Attributs
    private boolean fermee;
    private String nom;
    private int valeur;
    // Constructeur implicite
    Boite() {
        // ne fait rien
    }
    // Requêtes
    public String nom() {...}
    public int contenu() {...}
    public boolean fermee() {...}
    // Commandes
    public void remplir(int v) {...}
    public void ouvrir() {...}
    public void fermer() {...}
}
```

```
new Boite()
```



- Au moins un constructeur → pas d'ajout

Mot clé `this`

- **`this`** = instance courante

Rq. : appel non ciblé → **`this`**
- `valeur` ≡ **`this`**`.valeur`
- `fermee()` ≡ **`this`**`.fermee()`

```
public void remplir(int v) {
    if (this.fermee()) {
        ...
    }
    this.valeur = v;
}
```

- **`this(...)`** = appel d'un autre constructeur

```
class X {
    X(String s, int i) { ... }
    X(String s) {
        this(s, 0);
        ...
    }
}
```

→ /!\ 1ère ligne obligatoirement

Les classes sont aussi des objets

- À l'exécution :
  - classe représentée par un objet particulier
  - instance de la classe `Class`
  - état et comportement différents de ses instances
- *Attribut de classe*
  - champ **externe** aux instances
  - accessible à toutes les instances
  - initialisation au chargement de la classe
- *Méthode de classe*
  - service de (l'objet) classe
  - action **indépendante** des instances
  - ne peut pas utiliser **`this`** !

décrits dans le texte de la classe

- Déclaration des attributs de classe
  - **static**
- Initialisation des attributs de classe
  - en même temps que leur déclaration

accessible uniquement aux instances de X

```
class X {
    private static int nbInstances = 0;
    ...
}
```

  - ou dans un bloc statique (hors constructeur ou méthode)

```
class X {
    private static int nbInstances;
    static {
        nbInstances = 0;
    }
    ...
}
```

bloc statique permet d'initialiser les attributs statiques

code d'initialisation de la classe

- Définition des méthodes de classe
  - **static**

```
class X {
    private static int nbInstances = 0;
    public X() {
        nbInstances += 1;
    }
    public static int getNbInstances() {
        return nbInstances;
    }
    ...
}
```

- Appel de caractéristique statique

  → cible = une classe

- Cible non précisée

  → cible = classe englobante

```
class X {
    private static int instancesNb = 0;
    public static int getInstancesNb() {
        return instancesNb;
    }
    public X() {
        instancesNb += 1;
        ...
    }
    ...
}
```

Attribut de classe

par défaut la cible est `X`

cible obligatoire

```
class Y {
    ... m() {
        int z = X.getInstancesNb();
    }
}
```

Méthode de classe

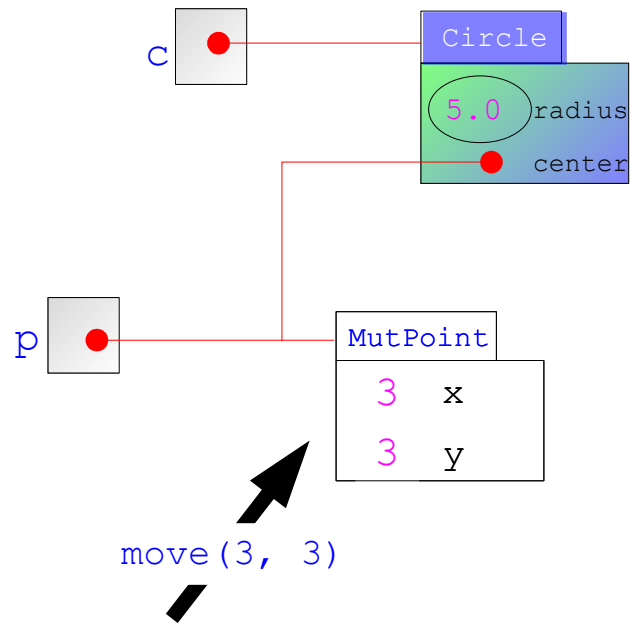
Objets mutables

- *Objet mutable* = peut modifier son état
- *Classe mutable* = instances mutables
- Exemple : `Boite`
  - méthodes `remplir`, `ouvrir` et `fermer`
- Contre-exemple : `String`
  - aucune méthode pour modifier l'état des instances

Attention à la mutabilité !

```
class Circle {
    private MutPoint center;
    private double radius;
    public Circle(MutPoint c, double r) {
        center = c;
        radius = r;
    }
    public MutPoint getCenter() {
        return center;
    }
    public void move(int dx, int dy) {
        center.move(dx, dy);
    }
    ...
}
```

```
MutPoint p = new MutPoint(0, 0);
Circle c = new Circle(p, 5.0);
p.move(3, 3);
```



Le cercle `c` a été déplacé bien qu'il n'ait pas reçu de message « déplace-toi »

La classe `MutPoint` est mutable

```
class MutPoint {
    private int x, y;
    public MutPoint(int x, int y) { this.x = x; this.y = y; }
    public int getX() { return x; }
    public int getY() { return y; }
    public void move(int dx, int dy) { x += dx; y += dy; }
    ...
}
```

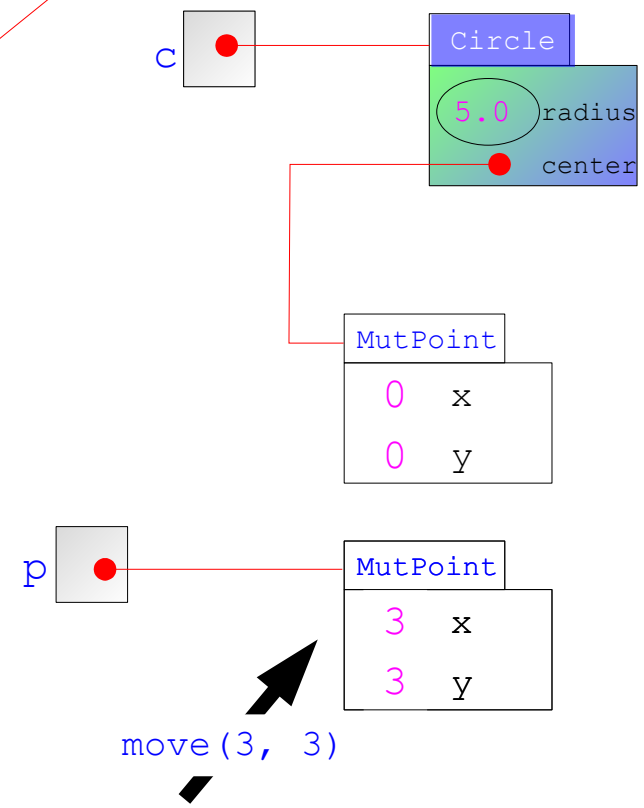
Solution 1

La classe `MutPoint` est dotée d'un constructeur réplicateur

```
class Circle {
    private MutPoint center;
    private double radius;
    public Circle(MutPoint c, double r) {
        center = new MutPoint(c);
        radius = r;
    }
    public MutPoint getCenter() {
        return new MutPoint(center);
    }
    public void move(int dx, int dy) {
        center.move(dx, dy);
    }
    ...
}
```

```
MutPoint p = new MutPoint(0, 0);
Circle c = new Circle(p, 5.0);
p.move(3, 3);
```

```
class MutPoint {
    public MutPoint(MutPoint p) {
        this.x = p.getX();
        this.y = p.getY();
    }
    ...
}
```



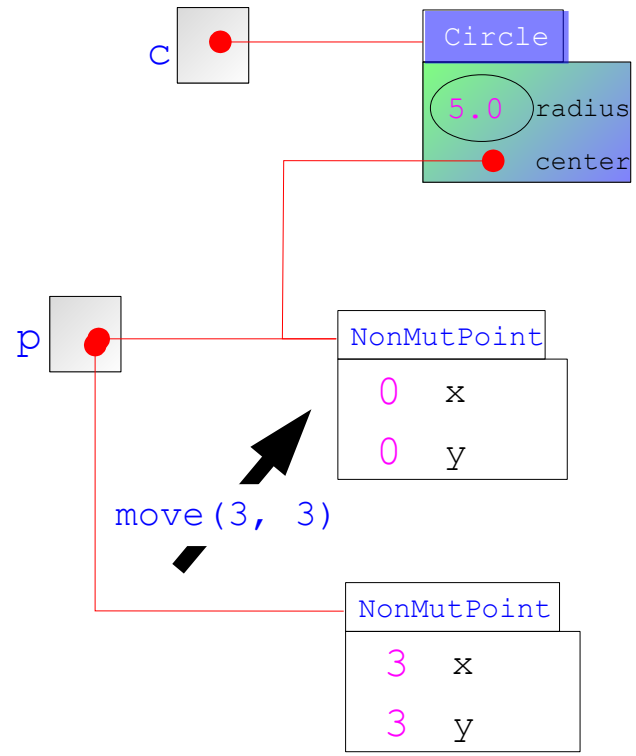
Le cercle `c`
n'a pas été déplacé

Solution 2

La classe `NonMutPoint` ne contient aucune commande permettant de modifier l'état de ses objets

```
class Circle {
    private NonMutPoint center;
    private double radius;
    public Circle(NonMutPoint c, double r) {
        center = c;
        radius = r;
    }
    public NonMutPoint getCenter() {
        return center;
    }
    public void move(int dx, int dy) {
        center = center.move(dx, dy);
    }
    ...
}
```

```
NonMutPoint p = new NonMutPoint(0, 0);
Circle c = new Circle(p, 5.0);
p = p.move(3, 3);
```




```
class NonMutPoint {
    private int x, y;
    public NonMutPoint(x, y) { this.x = x; this.y = y; }
    public int getX() { return x; }
    public int getY() { return y; }
    ...
    public NonMuTPoint move(int dx, int dy) { return new NonMutPoint(x+dx,y+dy); }
}
```

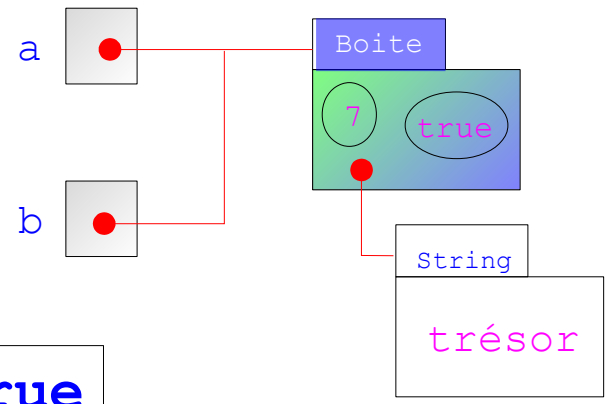
Tester l'identité de deux objets

- *Objets identiques* = même référence
  - les emplacements mémoire référencés sont identiques

Les entités `a` et `b` référencent le même objet
leurs valeurs sont identiques

```
Boite a = new Boite("trésor", 7);
Boite b = a;
```

Les objets référencés par `a` et `b` sont identiques

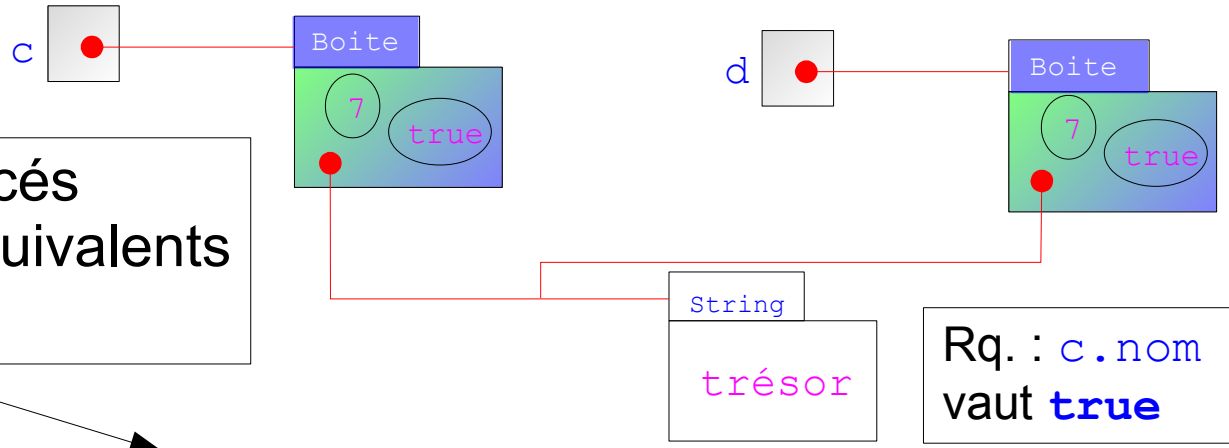


L'expression booléenne `a == b` vaut **`true`**

Tester l'équivalence de deux objets

- *Objets équivalents* = même classe, identiques ou d'état indistinguable

```
String nom = "trésor";
Boite c = new Boite(nom, 7);
Boite d = new Boite(nom, 7);
```



Les objets référencés par `c` et `d` sont équivalents (mais ici `c != d`)

Rq. : `c.nom == d.nom` vaut **`true`**

L'expression booléenne `c.equals(d)` vaut **`true`**

Méthode
`boolean equals(Object)`

- Présente dans toute classe
- Relation d'identité

```
public boolean equals(Object o) {
    return this == o;
}
```

- Remarque
  - Tout type référence est compatible avec `Object`
  - Une entité de type `Object` accepte tout objet

```
class X {
    int i;
    X(int v) {
        i = v;
    }
    public boolean equals(Object o) {
        return this == o;
    }
}
```

`a != b` et aussi `a.equals(b) == false`

ajouter une méthode `equals` dans `Y` permet d'y définir une relation d'équivalence moins fine que l'identité

```
class Y {
    int i;
    Y(int v) {
        i = v;
    }
    public boolean equals(Object o) {
        if ((o != null) && (o.getClass() == this.getClass())) {
            return (((Y) o).i == this.i);
        }
        return false;
    }
}
```

o est un objet

o et **this** sont instances de la même classe

o et **this** ont même état

`a != b` par contre `a.equals(b) == true`